



Zion

Developer Documentation

Version 1.0 • Church Website & Prayer Camp Booking System

Documentation philosophy: *This document explains not only what Zion contains, but why it was built this way, what was intentionally deferred, what went wrong during development, and how the architecture should evolve.*

Built to ship. Documented to evolve.

Contents

Zion --- Developer Documentation

Documentation philosophy

1. GET STARTED

1.1 What is Zion?

1.2 Technology stack

1.3 Why JavaScript instead of TypeScript?

1.4 Why vanilla CSS instead of Tailwind?

1.5 Project setup

1.6 Environment variables

1.7 Database setup

1.8 Prisma workflow

1.9 Seed data

1.10 Run the project

1.11 Production build

2. CONCEPTS

2.1 Application architecture

2.2 Authentication

2.3 Session identity

2.4 Users and roles

2.5 Authorization

2.6 Prayer requests

2.7 Site Content

2.8 Why JSON content?

2.9 Content editing flow

2.10 Caching and revalidation

2.11 Prayer camps

2.12 Booking philosophy for version 1

2.13 Important booking limitation

2.14 The future booking architecture

2.15 PWA

2.16 Styling philosophy

2.17 Fonts

2.18 Performance, SEO & Lighthouse status

2.19 Responsive behavior and known UI limitations

2.20 Upcoming-event banner CMS limitation

5.16 “Lighthouse is giving me poor scores”

6.17 “Images make the website slow”

5.18 “Videos are making the site heavy”

5.19 “The event banner covers my mobile dropdown”

3. GUIDES

3.1 Create a user

3.2 Create an administrator

3.3 Log in

3.4 Submit a prayer request

3.5 Edit website content

3.6 Book a prayer camp

3.7 Approve a booking

3.8 Add a new feature

4. REFERENCE

4.1 Repository structure

4.2 Route reference

Public routes

Authentication routes

User routes

Admin routes

4.3 Authentication reference

4.4 Component reference

Navbar

4.5 The Navbar authentication problem

4.6 Why the Navbar bug mattered

4.7 Prisma reference

4.8 Current database model

4.9 Database relationship

4.10 Server Action reference

4.11 Why Server Actions?

4.12 Content reference

4.13 PWA reference

4.14 Important PWA caveat

4.15 Static assets

4.16 Favicon

5. TROUBLESHOOTING

5.1 “The user is logged in, but the Navbar says logged out”

- 5.2 "I am authenticated but /admin redirects me"
- 5.3 "I changed content but the website still shows the old version"
- 5.4 "Prisma says a field is missing"
- 5.5 The updatedAt migration problem
- 5.6 Why resetting the database was acceptable during development
- 5.7 Seed data disappearing after reset
- 5.8 Server Action form errors
- 5.9 "The form submits but the server receives nothing"
- 5.10 Booking date problems
- 5.11 Booking capacity problems
- 5.12 "Prisma says booking or camp does not exist"
- 5.13 PWA does not work locally
- 5.14 PWA shows stale content
- 5.15 Images are not loading
- 5.16 Deployment problems

6. ENGINEERING NOTES

- 6.1 The most important design decision: ship the core first
- 6.2 Requirements vs first-version implementation
- 6.3 Why not build the perfect booking engine now?
- 6.4 Future booking state machine
- 6.5 Future camp management
- 6.6 Future email system
- 6.7 Future TypeScript migration
- 6.8 Future validation
- 6.9 Security principles
- 6.10 Performance philosophy
- 6.11 The developer's mental model
- 6.12 Debugging methodology
- 6.13 Development mistakes are architecture lessons
- 6.14 Current repository audit note
- 6.15 Recommended version 2 roadmap

Phase 1 --- Stabilize

Phase 2 --- Proper camp domain

Phase 3 --- Booking correctness

Phase 4 --- Communication

Phase 5 --- Documents

Phase 6 --- Type safety

Phase 7 --- Design system

Phase 8 --- PWA hardening

6.16 Developer checklist

7. THE ZION ENGINEERING PRINCIPLES

Principle 1 --- Ship the foundation before the complexity

Principle 2 --- Server owns truth

Principle 3 --- Content should not require code changes

Principle 4 --- Keep the architecture understandable

Principle 5 --- Temporary decisions must be documented

Principle 6 --- A bug is useful if it teaches the architecture

8. FINAL ARCHITECTURAL SUMMARY

Appendix A --- Quick command reference

Install

Development

Lint

Production build

Production server

Prisma generation

Development migration

Prisma Studio

Appendix B --- Quick file reference

Appendix C --- Definition of "done" for Zion v1

6.18 Known limitations at the end of v1

Zion --- Developer Documentation

Version: 1.0 / First Version\

Project: Zion Church Website & Prayer Camp Booking System\

Architecture: Next.js App Router + React + JavaScript + Prisma +

PostgreSQL + Auth.js\

Styling: Vanilla CSS / CSS Modules\

PWA: Serwist integration\

Deployment target: Vercel\

Repository: debrah-kobby/Zion

Documentation philosophy

This is not only a list of files and commands.

The purpose of this document is to explain **why** Zion was built the way it was built, what assumptions the first version makes, which requirements are complete, which are intentionally simplified, and where the architecture is expected to evolve.

If you are reading this as a future developer, the most important question is not:

"What does this file do?"

It is:

"Why does this file exist, why is it implemented this way, and what should I be careful not to break?"

That distinction matters because Zion is a first production-oriented version built under a real constraint: **time**.

The first version prioritizes:

1. A usable church website.
2. A clean and premium user experience.
3. Real authentication.
4. Role-based access.

5. A real database.
6. Admin content management.
7. Prayer requests.
8. A first working shape for prayer-camp booking.
9. A foundation that can support a significantly more sophisticated booking engine later.

The architecture therefore contains a few deliberate compromises. They are documented rather than hidden.

ZION

-
- ■ ■ ■ 1. GET STARTED
 - ■ ■ ■ What is Zion?
 - ■ ■ ■ Project setup
 - ■ ■ ■ Environment variables
 - ■ ■ ■ Database setup
 - ■ ■ ■ Seed data
 - ■ ■ ■ Run the project
 - ■ ■ ■ Production build
-
- ■ ■ ■ 2. CONCEPTS
 - ■ ■ ■ Application architecture
 - ■ ■ ■ Authentication
 - ■ ■ ■ Users & roles
 - ■ ■ ■ Prayer requests
 - ■ ■ ■ Prayer camps
 - ■ ■ ■ Bookings
 - ■ ■ ■ Site content
 - ■ ■ ■ Server Actions
 - ■ ■ ■ Caching & revalidation
 - ■ ■ ■ PWA
 - ■ ■ ■ Styling philosophy
-
- ■ ■ ■ 3. GUIDES
 - ■ ■ ■ Create a user
 - ■ ■ ■ Create an admin
 - ■ ■ ■ Add/edit site content
 - ■ ■ ■ Submit a prayer request
 - ■ ■ ■ Book a prayer camp
 - ■ ■ ■ Review a booking
 - ■ ■ ■ Add a new feature
-
- ■ ■ ■ 4. REFERENCE
 - ■ ■ ■ Repository structure
 - ■ ■ ■ Routes
 - ■ ■ ■ Database
 - ■ ■ ■ Server Actions
 - ■ ■ ■ Authentication
 - ■ ■ ■ Components
 - ■ ■ ■ Styling
 - ■ ■ ■ PWA
 - ■ ■ ■ Configuration
-
- ■ ■ ■ 5. TROUBLESHOOTING
 - ■ ■ ■ Authentication
 - ■ ■ ■ Navbar state
 - ■ ■ ■ Prisma
 - ■ ■ ■ Database migrations
 - ■ ■ ■ Server Actions
 - ■ ■ ■ PWA
 - ■ ■ ■ Assets
 - ■ ■ ■ Deployment
-

1. GET STARTED

1.1 What is Zion?

Zion is a modern church website with an authenticated member experience and administrative tooling.

The original project requirements described a system with three broad audiences:

Visitor

A visitor should be able to:

- Discover the church.
- Learn about the church.
- View ministries.
- View events.
- View sermons.
- Browse the gallery.
- Find contact information.
- Submit or access prayer-related functionality.
- Discover the prayer camp.
- Begin the booking process.

Registered user

A registered user should be able to:

- Create an account.
- Log in.
- Access their profile.
- Access a dashboard.
- Submit prayer requests.
- Manage their account.
- Participate in the prayer-camp workflow.

Administrator

An administrator should be able to:

- Access the admin dashboard.
- Manage users.
- Manage prayer requests.
- Manage website content.
- Review booking information.
- Approve or decline bookings where the booking subsystem is enabled.

The important architectural idea is that Zion is **not** just a brochure website.

It is a website + authenticated application + administrative system.

1.2 Technology stack

The original requirements proposed:

Area Requirement Zion choice

Frontend Next.js **Next.js**

UI React **React**

Styling Tailwind CSS **Vanilla CSS / CSS**

Modules

Language TypeScript **JavaScript**

Backend Next.js API or NestJS **Next.js Server**

Actions + Auth.js

Database PostgreSQL **PostgreSQL**

ORM Prisma **Prisma**

Authentication Not explicitly fixed **Auth.js / NextAuth**
credentials

PWA Not explicitly fixed **Servist**

Hosting Not explicitly fixed **Vercel-compatible**

deployment**

The current package uses Next.js 16.3.1, React 19.2.8, Prisma 7.9.1, PostgreSQL support through pg, Auth.js 5 beta, bcryptjs, and Serwist packages.

1.3 Why JavaScript instead of TypeScript?

This was a conscious first-version decision.

TypeScript was part of the original requirements, but the developer had not yet become comfortable with TypeScript.

That creates a legitimate engineering trade-off.

Why JavaScript was chosen

The priority was to understand and ship the application rather than introduce a second learning curve while simultaneously learning:

- Next.js App Router
- React Server Components
- Server Actions
- Prisma
- PostgreSQL
- Auth.js
- role-based authorization
- deployment
- PWA behavior

Adding TypeScript at the same time would have increased cognitive load substantially.

The reasoning was essentially:

<i>"I would rather build a complete application in JavaScript and</i>
<i>understand the architecture than use TypeScript superficially and</i>
<i>spend most of my time fighting the type system."</i>

That is a reasonable prototype/first-version trade-off.

The cost

The application gives up some compile-time guarantees.

For example, JavaScript does not automatically tell you that:

```
session.user.role
```

is expected to be one of:

```
VISITOR  
USER  
ADMIN
```

TypeScript could make those contracts explicit.

Future migration

The application can be migrated gradually.

A sensible migration order would be:

1. lib/
2. Server Actions
3. authentication utilities
4. reusable components
5. route pages
6. CSS-adjacent UI code

There is no requirement to rewrite the whole application in one pass.

1.4 Why vanilla CSS instead of Tailwind?

The original requirements specified Tailwind CSS.

Zion deliberately does not use Tailwind for the UI.

The reason is design control.

The project uses a heavily customized visual language:

- custom spacing
- layered sections
- image compositions
- typography combinations
- responsive navigation

- profile dropdowns
- admin layouts
- custom cards
- custom buttons
- decorative elements
- page-specific layouts

During development, Tailwind felt constraining because the developer wanted to reason about the actual CSS rather than constantly translating a design into utility classes.

The decision was therefore:

***Use CSS directly now. Preserve design freedom. Reconsider Tailwind*

*later if team scale makes utility-based styling valuable.***

What this gives us

Vanilla CSS provides:

- direct control
- easier experimentation
- no utility-class explosion
- clear visual ownership
- straightforward page-specific styling

What this costs

The downside is repetition.

As Zion grows, CSS can become harder to maintain because:

- naming needs discipline
- repeated values can drift
- responsive rules may be duplicated
- common patterns can become inconsistent

Future option

If the project eventually requires strict Tailwind usage, the UI can be migrated incrementally.

Do not rewrite business logic just because the styling system changes.

The important boundary is:

```
Business logic
  ↓
Components
  ↓
Styling
```

Changing the bottom layer should not require rewriting the top layers.

1.5 Project setup

Clone the repository and enter it:

```
git clone https://github.com/debrah-kobby/Zion.git
cd Zion
```

Install dependencies:

```
npm install
```

The project defines:

```
npm run dev
npm run build
npm run start
npm run lint
```

and runs Prisma generation as a `postinstall` step.

1.6 Environment variables

Zion requires environment configuration for the database and authentication.

A local `.env` should contain the required database connection string and authentication secret.

Conceptually:

```
DATABASE_URL="your-postgresql-connection-string"
AUTH_SECRET="your-long-random-secret"
```

Do **not** commit `.env`.

The environment variable names should remain stable because the Prisma and Auth.js configuration depend on them.

1.7 Database setup

Zion uses PostgreSQL through Prisma.

The Prisma schema defines a PostgreSQL datasource and generates the Prisma client into:

```
app/generated/prisma
```

The project contains migrations under:

```
prisma/migrations/
```

The existing migration history includes:

```
20260814071321_init
20260817084536_add_site_content
```

This is important: the migration history is part of the application's state. Treat it as production infrastructure, not temporary files.

1.8 Prisma workflow

For development, the normal mental model is:

```
schema.prisma
  ↓
Prisma migration
  ↓
PostgreSQL
  ↓
Generated Prisma Client
  ↓
Server-side application code
```

After changing a Prisma model:

```
npx prisma migrate dev --name describe-your-change
```

Then regenerate if necessary:

```
npx prisma generate
```

For inspecting the database:

```
npx prisma studio
```

1.9 Seed data

The seed file is:

```
prisma/seed.js
```

The seed populates website content such as:

- featured sermon
- contact phone
- homepage activity boxes
- prayer camp block
- ministries block
- about-page sermon
- events

It also contains prayer-camp seed logic.

This means the seed file is not merely "fake data."

It establishes the application's initial content model.

A developer changing SiteContent should consider whether the seed should change too.

1.10 Run the project

Start development:

```
npm run dev
```

Open:

```
http://localhost:3000
```

The application uses the Next.js App Router.

The primary application tree is:

```
app/  
components/  
lib/  
prisma/  
public/
```

1.11 Production build

Before deployment:

```
npm run lint
npm run build
```

Then:

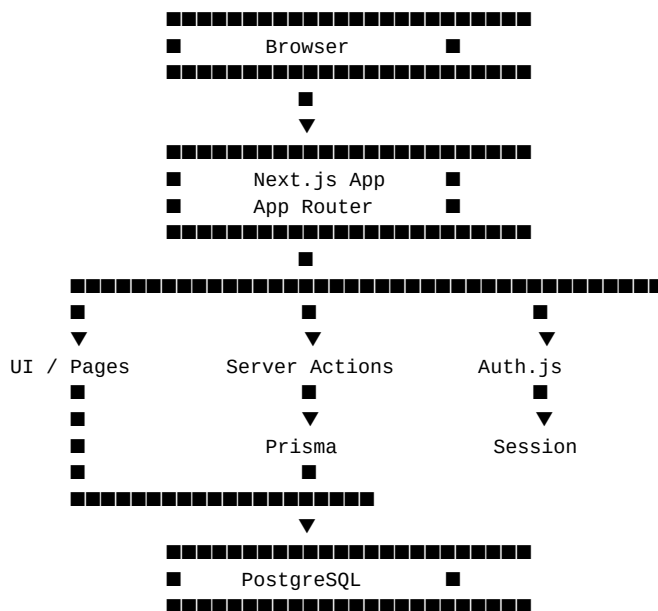
```
npm start
```

A successful local production build is an important checkpoint before pushing to production.

2. CONCEPTS

2.1 Application architecture

The high-level architecture is:



The application intentionally avoids introducing a separate NestJS backend.

Why?

Because the application does not currently need the operational complexity of:

```
Next.js
+
NestJS
+
separate API
```

- +
separate deployment
- +
separate authentication boundary

Next.js already provides server-side execution and Server Actions.

For this application's size, that is a strong simplification.

2.2 Authentication

Authentication is implemented using Auth.js / NextAuth.

The main authentication configuration lives in:

auth.js

Credentials authentication is used.

The login process is approximately:

```
User submits email/password
↓
Auth.js credentials provider
↓
Find user by email
↓
Compare password with bcrypt
↓
Return user identity
↓
JWT callback
↓
Session callback
↓
session.user.id
session.user.role
```

The credentials provider looks up the email in Prisma and verifies the password using bcrypt.

Passwords are therefore not compared as plain text.

2.3 Session identity

The session carries:

```
id
name
email
role
```

The JWT callback copies the user ID and role into the token.

The session callback then copies those values onto:

```
session.user
```

This matters because authorization code does not need to query the database for the user's role every time.

The session becomes the application's request-level identity.

2.4 Users and roles

The database currently defines:

```
VISITOR  
USER  
ADMIN
```

The intended meaning is:

VISITOR

Unauthenticated/public user.

USER

Authenticated member.

ADMIN

Authenticated administrator.

The role is stored on the User model.

```
User  
### id  
### name  
### email  
### password  
### role  
### createdAt  
### updatedAt
```

The default registered-user role is:

```
USER
```

2.5 Authorization

Authentication answers:

"Who are you?"

Authorization answers:

"Are you allowed to do this?"

Zion separates the two.

lib/auth-utils.js provides:

```
requireAuth()  
requireRole()
```

requireAuth():

1. calls auth()
2. checks for session.user
3. redirects unauthenticated users to /login

requireRole():

1. requires authentication
2. checks the session role
3. redirects unauthorized users to /dashboard

This is preferable to scattering role checks randomly throughout pages.

2.6 Prayer requests

Prayer requests have their own database model:

```
PrayerRequest  
■■■ id  
■■■ userId  
■■■ name  
■■■ department  
■■■ branch  
■■■ request  
■■■ note  
■■■ anonymous  
■■■ createdAt
```

A prayer request belongs to a user.

This gives us:

User 1 ■■■■■■■■■■ * PrayerRequest

The user relationship is enforced by Prisma.

2.7 Site Content

One of the most useful architectural decisions is the SiteContent model.

Instead of hardcoding every editable piece of church content into JSX, content can be represented as:

key → JSON value

The model is:

```
SiteContent
  id
  key
  value
```

The key is unique.

The value is JSON.

This creates a lightweight content-management layer without introducing a full CMS.

Examples include:

```
home_featured_sermon
contact_phone
home_fundays_boxes
home_prayer_camp_block
home_ministries_block
about_recent_sermon
events_upcoming_list
```

2.8 Why JSON content?

A relational table for every small content block would create a lot of models.

For example:

```
HomepageBanner
HomepageActivity
HomepageMinistryBlock
FeaturedSermon
```

```
ContactInformation
...
```

That is structurally cleaner for highly mature systems, but unnecessary for a first version.

JSON gives Zion:

- speed of development
- flexible content shapes
- fewer database tables
- easy admin editing
- fewer migrations

The trade-off is type safety.

A malformed JSON structure can break a page at runtime.

That is one area where TypeScript or schema validation would improve the application later.

2.9 Content editing flow

The content update action is:

```
app/actions/content.js
```

The flow is:

```
Admin opens content editor
↓
Editor loads SiteContent
↓
Admin modifies JSON-compatible value
↓
Server Action receives content ID + value
↓
Authenticate
↓
Check ADMIN role
↓
Prisma update()
↓
revalidatePath()
↓
Updated content becomes visible
```

The server action explicitly rejects:

- unauthenticated users
- non-admin users

- missing content IDs

This is important:

The UI hiding an admin button is not authorization.

The server must enforce authorization too.

2.10 Caching and revalidation

After updating content, Zion uses:

```
revalidatePath()
```

This is necessary because Next.js can cache/render data.

The application therefore follows:

```
Database changes
  ↓
Revalidate affected route
  ↓
Next.js knows the route needs fresh data
```

Without revalidation, an administrator could successfully update content in the database while the public page continues showing stale content.

2.11 Prayer camps

The prayer camp feature was intentionally kept simpler than the original requirements.

The original requirements imagined a full camp management system:

- multiple camps
- camp dates
- participant limits
- open/closed registration
- available capacity
- booking statuses
- booking IDs
- user booking history
- admin management

- confirmations
- potentially email

The first version does **not attempt to solve the entire problem**.

The reason is time.

The developer explicitly chose to establish the booking flow first and leave the complex scheduling/capacity engine for a later version.

This is an important architectural decision.

2.12 Booking philosophy for version 1

The booking form captures the core information needed for a camp registration:

Full name
Email
Phone
Gender
Date of birth
Emergency contact
Prayer request
Additional notes

The server action is:

`app/actions/booking.js`

The action validates required fields before proceeding.

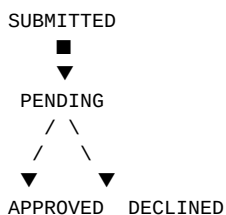
The admin status action is:

`app/actions/bookingAdmin.js`

It restricts status changes to:

APPROVED
DECLINED

The booking lifecycle conceptually becomes:



2.13 Important booking limitation

The booking engine is intentionally not the final booking architecture.

The UI contains a placeholder camp date in the current booking page while the real camp model is being wired into the application.

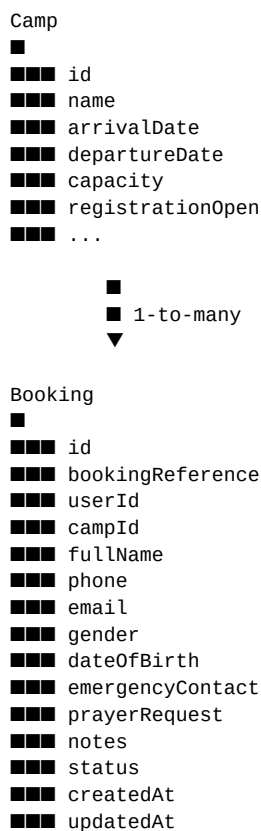
The seed code also contains camp-related persistence logic.

Therefore, the booking subsystem should be treated as **version 1 / transitional architecture**, not as the final camp-management engine.

The next version should make Camp and Booking first-class Prisma models and remove hardcoded camp information from the UI.

2.14 The future booking architecture

The mature version should look closer to:



Then capacity becomes:

```
available =
  camp.capacity
  -
  approved bookings
```

The final system should also protect against race conditions when two users attempt to claim the last available place simultaneously.

That means capacity should be enforced server-side, ideally transactionally.

2.15 PWA

Zion includes Serwist dependencies and a service worker:

```
app/sw.ts
```

The service worker uses:

```
Serwist
defaultCache
precacheEntries
skipWaiting
clientsClaim
```

The purpose is to move Zion toward installable Progressive Web App behavior.

A PWA gives the project the potential to behave more like an application on mobile devices.

The architecture is:

```
Next.js
+
Service Worker
+
Cache
+
Manifest
=
PWA
```

The current PWA setup should be considered an active foundation rather than a claim that every page is fully offline-capable.

Authenticated pages are especially important here: caching personalized dashboard/profile responses requires more care than caching public pages.

2.16 Styling philosophy

The application uses:

```
globals.css
*.module.css
```

rather than Tailwind utility classes.

There are page-specific styles such as:

```
page.module.css
profile.module.css
layout.module.css
AdminSidebar.module.css
```

This makes the CSS boundary visible.

A useful rule for future contributors:

<i>If a style is truly page-specific, keep it local. If a style</i>
<i>represents a site-wide design primitive, move it toward a shared</i>
<i>global/design-token layer.</i>

2.17 Fonts

The root layout uses Next.js font loading for several families including:

- Poppins
- Roboto Condensed
- Playfair Display
- Anton
- Oswald
- Alex Brush
- Give You Glory
- Edu NSW ACT Cursive

These are exposed through CSS variables.

This is deliberate because the visual identity uses different typographic roles rather than one universal font.

2.18 Performance, SEO & Lighthouse status

The first version intentionally prioritizes functionality and product completion over final production optimization.

As a result, a Lighthouse audit may currently report **below-target Performance and SEO scores**. This is a known v1 limitation, not something that should be interpreted as the application being architecturally complete.

Several optimization tasks were deliberately deferred because of the time constraint around the first release.

Current performance limitations

Some images in the repository are still relatively large source files. The application therefore carries more image weight than the final production build should.

The planned optimization workflow is:

```
Original image
  ↓
Squoosh
  ↓
Web-optimized image
  ↓
Next.js/public asset
```

The goal is to reduce:

- file size
- transfer time
- image decoding cost
- mobile bandwidth usage

The same principle applies to video.

The planned workflow is:

```
Original video
  ↓
HandBrake
  ↓
Compressed web-friendly video
  ↓
Production asset
```

Video compression should be done before treating large videos as production-ready assets.

SEO work still to be completed

The first version has not received the complete SEO pass that a production church website should eventually have.

The future SEO pass should include:

- page-specific metadata
- strong title/description coverage
- Open Graph metadata

- Twitter/X card metadata where appropriate
- canonical URLs
- sitemap
- robots configuration
- semantic heading review
- image alt text review
- structured data/schema markup where appropriate
- social sharing previews
- clean URL coverage
- indexability review

Until those tasks are completed, Lighthouse SEO should be treated as a **known optimization backlog item**.

Lighthouse is a measurement, not the product

The goal is not to “game” Lighthouse.

The goal is to fix the underlying causes:

```
Large assets
↓
Compression

Missing metadata
↓
SEO metadata

Unoptimized images
↓
Squoosh / responsive image strategy

Large videos
↓
HandBrake / appropriate delivery

Missing discoverability infrastructure
↓
Sitemap + robots + structured data
```

Once those changes are implemented, Lighthouse should be rerun on both mobile and desktop profiles.

2.19 Responsive behavior and known UI limitations

Zion is responsive, but the first version still contains a known mobile navigation layering bug.

Known bug: event banner overlays the mobile Navbar dropdown

On mobile, when a Navbar dropdown is open, the upcoming-events banner can visually appear **above the dropdown**.

Interestingly, this is not reproduced reliably when using desktop Chrome DevTools responsive mode.

The current behavior is therefore:

Desktop browser

→ dropdown layering appears correct

Desktop DevTools mobile emulation

→ dropdown layering appears correct

Actual mobile layout/device behavior

→ upcoming-event banner can cover dropdown

This is currently **unfixed**.

The likely area to investigate is the interaction between:

- stacking contexts
- z-index
- positioned ancestors
- transforms
- overflow rules
- mobile-specific layout behavior

Do not “fix” this by blindly increasing z-index.

A z-index only competes within the relevant stacking context. If two elements belong to different stacking contexts, a huge value on a child may not solve the real problem.

The correct debugging path is:

```
Navbar
↓
Dropdown ancestor
↓
position / z-index / transform / overflow
↓
Upcoming-event banner ancestor
↓
position / z-index / transform / overflow
↓
Mobile browser rendering
```

Future fix

The preferred fix is to identify the stacking-context boundary and establish an intentional layering hierarchy, for example:

```
Global page content
↓
Event banner
↓
Navbar
↓
Navbar dropdown
```

The exact CSS solution should be determined from the actual DOM rather than applying an arbitrary z-index: 999999.

2.20 Upcoming-event banner CMS limitation

The upcoming-event banner currently exists as part of the frontend experience, but its content is **not yet fully managed through the CMS**.

This is intentional v1 scope.

The CMS/content architecture should eventually expose the banner as an editable content object so an administrator can control:

- event title
- event date
- event time
- event description
- visibility
- CTA text
- CTA destination
- optional image/background
- start/end display window

The long-term model should be closer to:

```
Admin CMS
  ↓
Upcoming Event
  ↓
Database
  ↓
Server Component
  ↓
Event Banner
```

rather than:

```
Hardcoded frontend value
  ↓
Event Banner
```

This should be implemented after the core CMS/content model is stable.

5.16 “Lighthouse is giving me poor scores”

This is currently expected to some degree.

The first version did not spend its remaining development time on final optimization.

In particular:

Performance

→ image/video optimization still pending

SEO

→ full metadata/discoverability pass still pending

PWA

→ foundation exists, but production hardening is still pending

The correct response is not to hide the Lighthouse result.

Record it as a baseline.

Then fix the underlying issues and compare:

```
v1 baseline
  ↓
optimization pass
  ↓
Lighthouse rerun
  ↓
measured improvement
```

This makes performance work measurable rather than subjective.

6.17 “Images make the website slow”

Check the source files in `public/`.

Some images are larger than they need to be.

The planned workflow is:

1. Keep the original source outside the production asset set.
2. Open the image in Squoosh.
3. Choose an appropriate modern format/quality.
4. Export the compressed version.
5. Replace the production asset.
6. Rebuild.
7. Run Lighthouse again.

Do not blindly compress everything to the lowest quality.

The goal is:

smallest acceptable file

not:

5.18 “Videos are making the site heavy”

Videos are media assets, not ordinary images.

Use HandBrake to create web-friendly versions.

The production strategy should consider:

- resolution
- bitrate
- codec
- duration
- whether autoplay is actually necessary
- whether a poster image can be used
- whether the video should be lazy-loaded
- whether a CDN/video platform is more appropriate at scale

Large raw videos should not be treated as optimized web assets.

5.19 “The event banner covers my mobile dropdown”

This is a known unresolved v1 bug.

Do not treat it as an authentication issue.

The Navbar session can be completely correct while the dropdown is visually obscured.

Investigate:

z-index
position
overflow
transform
stacking context

on both:

Navbar
Upcoming event banner

and their positioned ancestors.

The fact that DevTools desktop emulation does not reproduce the problem is useful evidence: test on the actual target mobile browser as well.

3. GUIDES

3.1 Create a user

The registration flow lives under:

`/app/signup`

The user submits the registration form.

The server-side authentication action:

`app/actions/auth.js`

is responsible for validating the request and creating the account.

The important rule is:

Never create a user by directly inserting a plain-text password.

Passwords must be hashed with bcrypt.

The resulting database record should contain:

email
name
hashed password
role
timestamps

The normal user role is:

USER

3.2 Create an administrator

Do not allow the public signup form to freely choose:

ADMIN

That would be a privilege-escalation vulnerability.

Admin creation should be controlled through:

- a protected seed
- a trusted database operation
- a dedicated admin-management flow

The role belongs to the server-side authorization boundary.

3.3 Log in

Login uses the credentials provider.

The mental model:

```
Login form
  ↓
Auth.js
  ↓
Prisma User lookup
  ↓
bcrypt password verification
  ↓
JWT
  ↓
Session
```

A failed password should not reveal whether the email exists.

This prevents unnecessary account enumeration.

3.4 Submit a prayer request

The public/user-facing prayer request route is:

`/prayer-request`

The server action is:

`app/actions/prayer.js`

The request should be associated with the authenticated user.

The database relationship means administrators can later retrieve requests without relying on fragile client-side state.

3.5 Edit website content

Admin content lives under:

/admin/content

The workflow:

1. Log in as admin.
2. Open the admin dashboard.
3. Open content management.
4. Select the content entry.
5. Edit its value.
6. Save.
7. Server validates admin privileges.
8. Prisma updates SiteContent.
9. Relevant paths are revalidated.

This is intentionally simple.

It is closer to a lightweight structured CMS than a full WordPress-style editor.

3.6 Book a prayer camp

The booking page is:

/camp/book

The form submits to:

createBooking()

from:

app/actions/booking.js

The first version validates the core personal information.

The important principle is:

<i>Validation must happen on the server even if the browser also</i>
<i>validates the form.</i>

Client-side validation is for user experience.

Server-side validation is for correctness and security.

3.7 Approve a booking

The admin booking area is:

```
/admin/bookings
```

The status update action is:

```
updateBookingStatus()
```

from:

```
app/actions/bookingAdmin.js
```

Only admins can perform the operation.

The server accepts only:

```
APPROVED  
DECLINED
```

After updating the database, the application revalidates:

```
/admin/bookings  
/dashboard  
/profile
```

This is important because booking status can appear in more than one user interface.

3.8 Add a new feature

Before adding a feature, ask four questions:

1. Is this public or authenticated?

Public:

```
app/something/
```

Authenticated:

```
requireAuth()
```

Admin:

```
requireRole("ADMIN")
```

2. Does it need persistence?

If yes:

```
Prisma schema
↓
migration
↓
server action
↓
UI
```

3. Does it change cached content?

If yes, consider:

```
revalidatePath()
```

4. Does the browser really need to own the logic?

Prefer:

```
Server Component
```

or:

```
Server Action
```

when possible.

Only introduce "use client" when client-side interactivity actually requires it.

4. REFERENCE

4.1 Repository structure

The repository currently follows this broad structure:

```
Zion/
■
■ ■ ■ app/
■   ■ ■ ■ about/
■   ■ ■ ■ actions/
■   ■ ■ ■ admin/
■   ■ ■ ■ api/
■   ■ ■ ■ camp/
■   ■ ■ ■ dashboard/
■   ■ ■ ■ events/
■   ■ ■ ■ gallery/
■   ■ ■ ■ give/
■   ■ ■ ■ login/
■   ■ ■ ■ ministries/
■   ■ ■ ■ prayer-request/
■   ■ ■ ■ profile/
■   ■ ■ ■ sermons/
```

- ■■■■ signup/
- ■■■■ favicon.ico
- ■■■■ globals.css
- ■■■■ layout.js
- ■■■■ page.jsx
- ■■■■ sw.ts
-
- ■■■■ components/
- ■■■■ AuthProvider.jsx
- ■■■■ Footer.jsx
- ■■■■ HomeClient.jsx
- ■■■■ Navbar.jsx
-
- ■■■■ lib/
- ■■■■ auth-utils.js
- ■■■■ prisma.js
-
- ■■■■ prisma/
- ■■■■ migrations/
- ■■■■ schema.prisma
- ■■■■ seed.js
-
- ■■■■ public/
- ■■■■ images
- ■■■■ other static assets
-
- ■■■■ auth.js
- ■■■■ next.config.mjs
- ■■■■ package.json
- ■■■■ jsconfig.json

4.2 Route reference

Public routes

/

Homepage.

/about

Church information.

/events

Events.

/gallery

Gallery.

/ministries

Ministries.

/sermons

Sermons.

/give

Giving.

/prayer-request

Prayer requests.

/camp

Prayer camp landing page.

/camp/book

Prayer camp booking.

Authentication routes

/login
/signup

User routes

/profile
/dashboard

These should require authentication.

Admin routes

/admin
/admin/bookings
/admin/content
/admin/prayer-requests
/admin/users

These should require administrator privileges.

4.3 Authentication reference

Important files:

auth.js
lib/auth-utils.js
components/AuthProvider.jsx
app/actions/auth.js

`auth.js`

Owns Auth.js configuration.

`AuthProvider.jsx`

Provides the client-side session context through:

`SessionProvider`

`auth-utils.js`

Owns reusable authorization guards.

`actions/auth.js`

Owns authentication-related server operations such as signup/login support.

4.4 Component reference

Navbar

`components/Navbar.jsx`

The Navbar is one of the most stateful components.

It handles:

- navigation
- responsive menu
- profile menu
- login state
- user initials
- admin visibility
- logout
- mobile interactions

Because it depends on authentication, it is intentionally a client component.

4.5 The Navbar authentication problem

One of the memorable bugs during development was the profile indicator showing:

```
"Not logged in"
```

even when the user was clearly authenticated and already on their profile.

This looked like an authentication failure.

It was not necessarily an authentication failure.

The deeper problem was **multiple representations of authentication state**.

The application had:

```
server session
```

and:

```
client session
```

These can become temporarily out of sync during navigation/hydration.

The lesson:

```
Authentication is not only a backend problem. It is also a UI  
synchronization problem.
```

The eventual approach was to make the Navbar's authenticated rendering depend on the session state rather than trying to infer authentication from the current URL.

The Navbar now uses the session/user information to determine:

```
isLoggedIn  
isAdmin  
userInitial
```

rather than assuming:

```
/profile === logged in
```

This distinction is crucial.

A route tells you where the user is.

A session tells you who the user is.

Those are different facts.

4.6 Why the Navbar bug mattered

A bug like this can be dangerous because developers may "fix" it by doing something like:

```
if pathname === "/profile"
  assume logged in
```

That is wrong.

A user can access or navigate to routes in many different ways.

Authorization should always use:

session

not:

URL

The URL is presentation state.

The session is identity state.

4.7 Prisma reference

The Prisma client is centralized in:

lib/prisma.js

This avoids creating arbitrary Prisma clients throughout the application.

The general rule should be:

```
UI
↓
Server Action / Server Component
↓
lib/prisma.js
↓
PostgreSQL
```

Do not query PostgreSQL directly from browser components.

4.8 Current database model

The current checked-in Prisma schema defines:

Role

VISITOR
USER
ADMIN

User

id
name
email
password
role
createdAt
updatedAt

SiteContent

id
key
value

PrayerRequest

id
userId
name
department
branch
request
note
anonymous
createdAt

4.9 Database relationship

The primary current relationship is:

User
■
■■■■■■■■■■■■■■■■■■■■< PrayerRequest

This allows one user to have multiple prayer requests.

4.10 Server Action reference

The application currently organizes server-side mutations under:

```
app/actions/
```

The repository contains:

```
auth.js
booking.js
bookingAdmin.js
content.js
logout.js
prayer.js
```

This is a good boundary.

It keeps mutation logic away from presentational components.

4.11 Why Server Actions?

For Zion, Server Actions eliminate unnecessary API boilerplate.

Instead of:

```
React
↓
fetch("/api/update-content")
↓
API route
↓
validation
↓
database
```

the application can use:

```
React form
↓
Server Action
↓
validation
↓
database
```

This is especially useful for internal application operations such as:

- admin content updates
- prayer requests
- authentication workflows
- booking mutations

A separate REST API can still be introduced later if a mobile app,

external integration, or third-party client requires it.

4.12 Content reference

The seed data currently demonstrates content keys such as:

```
home_featured_sermon
contact_phone
home_fundays_boxes
home_prayer_camp_block
home_ministries_block
about_recent_sermon
events_upcoming_list
```

A useful convention for future keys is:

```
{page}_{section}_{purpose}
```

For example:

```
home_hero
home_featured_sermon
about_history
about_pastor
contact_information
events_upcoming_list
```

Avoid vague keys such as:

```
content1
stuff
homepageData
```

The database should explain itself.

4.13 PWA reference

PWA-related files/dependencies include:

```
app/sw.ts
@serwist/next
serwist
```

The service worker uses Serwist's default runtime cache.

The service worker currently enables:

```
skipWaiting
clientsClaim
```

This helps new service-worker versions take control more aggressively.

4.14 Important PWA caveat

Do not assume:

"PWA installed" = "everything works offline."

Authenticated application pages are personalized.

For example:

`/dashboard`
`/profile`

can contain user-specific information.

Caching those responses carelessly could expose the wrong cached state or display stale information.

The safest future direction is:

- aggressively cache public assets/pages
- carefully handle authenticated data
- define an explicit offline experience
- avoid caching sensitive personalized responses as generic public

content

4.15 Static assets

Static assets live under:

`public/`

Examples include:

- church imagery
- camp imagery
- logos
- profile images
- videos
- background assets

A public asset:

`public/churchhero2.avif`

is referenced as:

```
/churchhero2.avif
```

Do not use filesystem paths in JSX.

4.16 Favicon

The repository contains:

```
app/favicon.ico
```

With the Next.js App Router, this convention is automatically recognized as the site's favicon.

This is preferable to manually adding a favicon `<link>` when the framework convention already handles it.

5. TROUBLESHOOTING

5.1 "The user is logged in, but the Navbar says logged out"

First check:

```
AuthProvider  
Navbar session hook  
auth.js session callback
```

Do not immediately change routing logic.

The debugging question is:

```
Does the server know the user?  
  ↓  
Does the client know the user?  
  ↓  
Does Navbar render from the client session?
```

If the profile page works but the Navbar does not, suspect a client/session synchronization issue before suspecting the database.

5.2 "I am authenticated but /admin redirects me"

Check the session:

```
session.user.role
```

The role must be:

```
ADMIN
```

Remember:

```
authenticated ≠ administrator
```

A normal user should be authenticated but still fail the admin authorization check.

5.3 "I changed content but the website still shows the old version"

Check that the Server Action calls:

```
revalidatePath()
```

The current content action revalidates the admin content routes after an update.

If a new content route is introduced, remember to revalidate the route that consumes the changed data.

5.4 "Prisma says a field is missing"

This usually means one of three things:

1. The schema changed but the database did not.
2. The migration changed but Prisma Client is stale.
3. The generated client does not match the schema.

Try:

```
npx prisma generate
```

Then inspect:

```
npx prisma studio
```

For development migrations:

```
npx prisma migrate dev
```

5.5 The updatedAt migration problem

One of the real development problems was adding a required:

updatedAt

field to existing User records.

The database already contained rows.

A required field with:

@updatedAt

cannot always be introduced into existing data without considering how those rows will receive their initial values.

This is a classic migration problem.

The lesson:

Prisma schema changes are database migrations, not just JavaScript

changes.

5.6 Why resetting the database was acceptable during development

At the time, the database contained dummy development users.

A reset was therefore acceptable.

It would **not** be an acceptable default solution once the application contains real church/member data.

The principle is:

Dummy development data
→ reset can be acceptable

Real production data

→ never casually reset

5.7 Seed data disappearing after reset

Another development lesson was that resetting the database also removed the `SiteContent` records.

That is exactly what a reset is expected to do.

The fix was to ensure the seed script recreated the initial content.

This demonstrates why seeding matters.

A reproducible application should be able to go:

```
Empty database
  ↓
Migration
  ↓
Seed
  ↓
Usable development environment
```

5.8 Server Action form errors

Auth forms use client-side wrappers with React form state/status handling.

A Server Action used with `useActionState` must match the expected calling shape:

```
(previousState, formData)
```

A common mistake is accidentally defining:

```
(formData)
```

and then wondering why the values are wrong.

When debugging a form action, first inspect:

```
action signature
form action binding
FormData field names
returned state shape
```

The names must agree.

5.9 "The form submits but the server receives nothing"

Check:

```
name="email"  
name="password"  
name="fullName"
```

The server action reads:

```
formData.get("email")  
formData.get("password")  
...
```

The HTML field name and the server-side FormData key must match exactly.

5.10 Booking date problems

The first booking UI has a transitional hardcoded camp date.

That is intentional for the first version.

However, it should not remain in the final architecture.

The mature system should obtain camp information from:

Camp

in PostgreSQL.

Never build the final system around:

```
const campStartDate = "2026-08-20";  
const campEndDate = "2026-08-24";
```

The database should own event/camp truth.

5.11 Booking capacity problems

The current first version should not be described as a complete capacity-management engine.

The original requirements included:

```
maximum participants  
available spaces
```

```
open registration
close registration
```

Those concepts require a proper camp model and transactional capacity logic.

A future implementation should not simply do:

```
count bookings
if count < capacity
  create booking
```

without thinking about concurrency.

Two simultaneous requests can both observe the same remaining capacity.

The final implementation needs an atomic/transaction-safe strategy.

5.12 "Prisma says booking or camp does not exist"

This is an important repository consistency check.

The current `prisma/schema.prisma` visible in the repository defines `User`, `SiteContent`, and `PrayerRequest`, while booking-related server actions and the seed script reference `Booking/Camp`.

If this state is reproduced locally, the schema and application code are out of sync.

Do not solve this by randomly editing generated Prisma files.

Instead:

1. Inspect the intended schema.
2. Add the missing models to `prisma/schema.prisma`.
3. Create a migration.
4. Run Prisma generation.
5. Re-run the build.
6. Test the booking flow.

Generated Prisma output should never be treated as the source of truth.

The source of truth is:

```
prisma/schema.prisma
```

5.13 PWA does not work locally

Service workers can behave differently in development and production.

Check:

```
app/sw.ts
next.config.mjs
service worker registration
browser Application → Service Workers
```

Also remember that a stale service worker can make a development bug look like a code bug.

When testing PWA behavior:

1. Unregister the service worker.
 2. Clear site data/cache.
 3. Restart the app.
 4. Rebuild.
 5. Test again.
-

5.14 PWA shows stale content

This is one of the most annoying PWA debugging problems.

The browser may be showing:

cached application

rather than:

current application

When debugging:

```
DevTools
→ Application
→ Service Workers
→ Unregister
```

Then clear storage.

Never spend an hour debugging JSX when the browser is serving an old cached build.

5.15 Images are not loading

For local assets, verify:

`public/`

For example:

`public/ZionLogo1.webp`

should be referenced as:

`/ZionLogo1.webp`

not:

`/public/ZionLogo1.webp`

5.16 Deployment problems

The project is designed to be compatible with Vercel-style Next.js deployment.

The deployment environment must have:

`DATABASE_URL`
`AUTH_SECRET`

and any future external-service variables.

Before deploying:

```
npm run lint
npm run build
```

Then verify:

- authentication
- database connection
- admin authorization
- content editing
- profile page
- dashboard
- prayer request
- booking
- PWA behavior

6. ENGINEERING NOTES

6.1 The most important design decision: ship the core first

The original requirements were larger than the time available.

Trying to implement everything at once would have created:

```
half-finished booking
+
half-finished CMS
+
half-finished authentication
+
half-finished PWA
```

Instead, the project prioritized a functioning foundation.

That is the correct trade-off for a first version.

A first version is not supposed to solve every future problem.

It is supposed to make the next version easier.

6.2 Requirements vs first-version implementation

Requirement	First version
Modern church website	Implemented
Responsive design	Implemented
Homepage	Implemented
About	Implemented
Ministries	Implemented
Events	Implemented
Sermons	Implemented
Gallery	Implemented
Contact	Implemented
User registration	Implemented
User login	Implemented

User profile Implemented
User dashboard Implemented
Roles Implemented
Admin dashboard Implemented
Admin users Implemented
Admin prayer requests Implemented
Content management Implemented
Prayer camp landing page Implemented
Booking UI First-version implementation
Booking status workflow First-version implementation
Dynamic camp management **Future expansion**
Capacity enforcement **Future expansion**
Multiple camp scheduling **Future expansion**
Automated email confirmation **Future expansion**
Downloadable booking confirmation **Future expansion**
Full offline PWA **Future expansion**
TypeScript **Future migration**
Tailwind **Optional future migration**

6.3 Why not build the perfect booking engine now?

Because the perfect booking engine is not just a form.

A real camp system eventually has to answer:

How many camps exist?

Who can book?

Can someone book twice?

What happens when capacity is reached?

What happens if an admin closes registration?

What happens if an approved booking is cancelled?

Can dates overlap?

Can a person modify their booking?

Does an emergency contact need validation?

What happens if two users book the final space simultaneously?

What emails are sent?

What happens after approval?

What happens after decline?

Can admins export the list?

Can users download confirmation?

What information should be visible to other users?

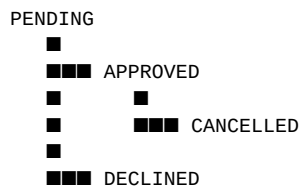
What information is sensitive?

That is a real domain model.

It should be implemented deliberately rather than rushed.

6.4 Future booking state machine

A more mature system could use:



Potentially:

WAITLISTED

could also be introduced when capacity is reached.

The status should be represented as an enum rather than arbitrary strings.

6.5 Future camp management

A proper admin camp screen should eventually allow:

- Create camp
- Edit camp
- Open registration
- Close registration
- Set capacity
- View capacity
- View participants
- Export participants
- Archive camp

The public camp page should then query the database rather than contain hardcoded dates.

6.6 Future email system

The original requirements mark confirmation email as optional.

The first version does not depend on an email provider.

That is intentional.

Email introduces another operational dependency:

```
Email provider
+
credentials
+
templates
+
delivery failures
+
rate limits
+
bounce handling
```

It should be introduced after the core booking state machine is stable.

6.7 Future TypeScript migration

A sensible migration strategy is:

```
JavaScript application
↓
Type the data boundaries
↓
Type Server Actions
↓
Type Prisma result usage
↓
Type components
↓
Type routes
```

The most valuable thing to type first is not every UI component.

It is the data crossing boundaries.

For example:

```
Database → Server Action
Server Action → UI
Session → Authorization
```

Those are the places where incorrect assumptions hurt most.

6.8 Future validation

The application currently performs manual validation.

As the domain grows, introduce a schema-validation library such as Zod or equivalent.

The ideal future flow becomes:

```
FormData
↓
Validation schema
↓
Normalized data
↓
Authorization
↓
Database transaction
```

This becomes particularly valuable for booking because the form contains personal information and dates.

6.9 Security principles

The application should always preserve these rules.

Rule 1 --- Never trust the browser

A hidden admin button does not make an operation secure.

Authorization belongs on the server.

Rule 2 --- Never store plaintext passwords

Use bcrypt or a secure password hashing mechanism.

Rule 3 --- Never trust client-provided roles

A client should never be able to submit:

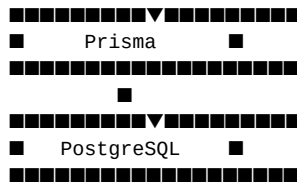
```
role=ADMIN
```

and become an administrator.

Rule 4 --- Validate server-side

Client validation is UX.

Server validation is security/correctness.



When something breaks, locate the layer first.

Do not immediately rewrite everything.

6.12 Debugging methodology

When a feature fails, ask:

Is the UI wrong?

Inspect the component.

Is the client state wrong?

Inspect hooks/session state.

Is the request wrong?

Inspect form names and Server Action arguments.

Is authorization wrong?

Inspect:

```
session.user.role
```

Is the data wrong?

Inspect Prisma/database.

Is the page stale?

Inspect:

```
revalidatePath  
cache  
PWA/service worker
```

Is production different?

Compare environment variables and deployment configuration.

This prevents random debugging.

6.13 Development mistakes are architecture lessons

The bugs encountered during development are valuable documentation.

Examples:

Navbar authentication mismatch

Lesson:

`URL ≠ identity`

Prisma required field migration failure

Lesson:

`schema change ≠ harmless code change`

Reset deleting content

Lesson:

`database state must be reproducible`

Server Action argument mismatch

Lesson:

`framework conventions are part of the API contract`

Hardcoded camp date

Lesson:

`temporary UI state must eventually move to domain data`

These are not just mistakes.

They show where the architecture needs strong boundaries.

6.14 Current repository audit note

The checked-in repository currently contains a few signs of the transition between the first version and the next booking iteration.

For example:

- `booking.js` references booking persistence.
- `bookingAdmin.js` references booking persistence.
- `prisma/seed.js` contains camp persistence.
- the current visible `schema.prisma` defines `User`, `SiteContent`,

and `PrayerRequest`.

That means the repository should be checked for schema/application synchronization before treating the booking subsystem as production-complete.

This documentation intentionally does **not** hide that discrepancy.

Good documentation should tell the next developer what the code actually does, not what the original specification hoped it would do.

6.15 Recommended version 2 roadmap

Phase 1 --- Stabilize

- Align Prisma schema with booking actions.
- Confirm migrations.
- Run a clean database setup.
- Run lint.
- Run production build.
- Test authentication end-to-end.

Phase 2 --- Proper camp domain

Create:

Camp
Booking

models.

Add:

capacity
registrationOpen
status
bookingReference

and proper relations.

Phase 3 --- Booking correctness

Implement:

- duplicate booking protection
- capacity enforcement
- registration windows
- booking cancellation
- transaction safety
- admin editing

Phase 4 --- Communication

Add:

- booking confirmation email
- approval email
- decline email
- reminder email

Phase 5 --- Documents

Add:

- downloadable booking confirmation
- printable confirmation
- admin export

Phase 6 --- Type safety

Begin JavaScript → TypeScript migration.

Phase 7 --- Design system

Decide whether to:

- keep CSS Modules
- introduce CSS variables/design tokens
- migrate to Tailwind

The styling decision should be based on project scale, not ideology.

Phase 8 --- PWA hardening

Implement:

- explicit manifest
 - install experience
 - offline fallback
 - public-page caching
 - careful authenticated-page behavior
 - service-worker update strategy
-

6.16 Developer checklist

Before merging a feature:

```
[ ] Does it need authentication?
[ ] Does it need authorization?
[ ] Is server-side validation present?
[ ] Does it write to the database?
[ ] Does Prisma need a migration?
[ ] Does generated Prisma Client need regeneration?
[ ] Does the route need revalidation?
[ ] Is the component unnecessarily client-side?
[ ] Does the feature expose sensitive data?
[ ] Does it work on mobile?
[ ] Does it work after refresh?
[ ] Does it work after logout/login?
[ ] Does it work for USER?
[ ] Does it work for ADMIN?
[ ] Does it fail safely for unauthorized users?
[ ] Does `npm run lint` pass?
[ ] Does `npm run build` pass?
[ ] Does the feature behave correctly with the PWA/service worker?
```

7. THE ZION ENGINEERING PRINCIPLES

Principle 1 --- Ship the foundation before the complexity

A simple booking system that can evolve is better than a sophisticated booking system that never ships.

Principle 2 --- Server owns truth

The server owns:

- identity
- authorization

- validation
- database writes
- booking status
- capacity

The browser owns:

- interaction
- presentation
- temporary UI state

Principle 3 --- Content should not require code changes

The `SiteContent` abstraction exists specifically to reduce the need to edit JSX whenever church content changes.

Principle 4 --- Keep the architecture understandable

Do not introduce an abstraction merely because it is technically impressive.

A small application benefits from:

```
Next.js
+
Prisma
+
PostgreSQL
+
Auth.js
+
Server Actions
```

without automatically needing:

```
NestJS
+
GraphQL
+
Redis
+
message queues
+
microservices
```

Principle 5 --- Temporary decisions must be documented

Hardcoded camp dates, JavaScript, CSS instead of Tailwind, and simplified booking are not accidents.

They are decisions made under first-version constraints.

Documenting them prevents future developers from mistaking a deliberate trade-off for negligence.

Principle 6 --- A bug is useful if it teaches the architecture

The Navbar session problem taught the difference between navigation state and authentication state.

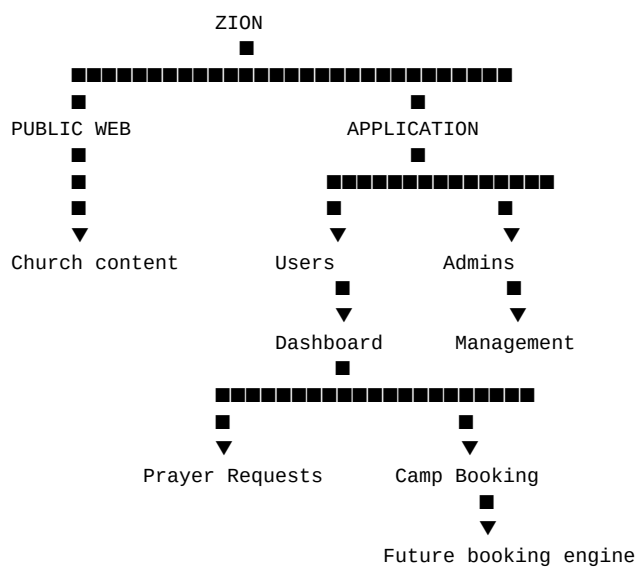
The Prisma migration problem taught the difference between schema design and database migration.

The seed/reset problem taught the importance of reproducibility.

These lessons should survive even after the original bugs disappear.

8. FINAL ARCHITECTURAL SUMMARY

Zion is best understood as:



The first version is intentionally not the final system.

It is the foundation.

The important pieces are already separated:

```
UI
↓
Server Actions
↓
Authorization
↓
Prisma
```

↓
PostgreSQL

That separation is what makes future growth possible.

The next major architectural milestone is not "add more pages."

It is to make the domain models---especially **Camp** and

Booking---fully first-class and then build the advanced booking workflow on top of them.

Appendix A --- Quick command reference

```
# Install
npm install

# Development
npm run dev

# Lint
npm run lint

# Production build
npm run build

# Production server
npm start

# Prisma generation
npx prisma generate

# Development migration
npx prisma migrate dev --name your_change

# Prisma Studio
npx prisma studio
```

Appendix B --- Quick file reference

```
Authentication
→ auth.js

Authorization
→ lib/auth-utils.js

Database client
→ lib/prisma.js

Database schema
→ prisma/schema.prisma

Seed
→ prisma/seed.js

Auth server actions
→ app/actions/auth.js

Booking server actions
```

→ app/actions/booking.js

Booking admin actions
→ app/actions/bookingAdmin.js

Content server actions
→ app/actions/content.js

Prayer actions
→ app/actions/prayer.js

Navbar
→ components/Navbar.jsx

Session provider
→ components/AuthProvider.jsx

Global layout
→ app/layout.js

PWA service worker
→ app/sw.ts

Favicon
→ app/favicon.ico

Static assets
→ public/

Admin area
→ app/admin/

User dashboard
→ app/dashboard/

User profile
→ app/profile/

Prayer camp
→ app/camp/

Appendix C --- Definition of "done" for Zion v1

Zion v1 is considered complete when:

- ✓ Visitors can navigate the church website.
- ✓ Users can register.
- ✓ Users can authenticate.
- ✓ Users have roles.
- ✓ Users can access protected areas.
- ✓ Admins have a protected admin area.
- ✓ Admins can manage supported site content.
- ✓ Users can submit prayer requests.
- ✓ The prayer-camp experience exists.
- ✓ The first booking workflow exists.
- ✓ The application has a real PostgreSQL database.
- ✓ Prisma manages persistence.
- ✓ Authentication is server-backed.
- ✓ The application has a PWA foundation.
- ✓ The application can be deployed.
- ✓ The developer understands why the architecture looks the way it does.

The last item is arguably the most important.

A project is much easier to maintain when the developer who built it can

explain **why** each major decision was made.

Zion v1 --- built to ship, documented to evolve.

6.18 Known limitations at the end of v1

The following items are intentionally recorded as **not yet complete** rather than being presented as bugs that have already been solved:

Item	Status	Reason / next step
Full booking engine	Deferred	Replace transitional booking logic with first-class Camp/Booking domain models
Camp capacity enforcement	Deferred	Requires transactional capacity logic
Booking emails	Deferred	Add an email provider and event-driven notifications
Full CMS for upcoming-event banner	Deferred	Move banner data into SiteContent or a dedicated Event model
Mobile Navbar/event-banner layering	Known bug	Investigate stacking contexts and mobile browser behavior
Image optimization	Deferred	Process production images through Squoosh
Video optimization	Deferred	Compress production videos with HandBrake
Lighthouse Performance	Below target	Asset optimization and rendering improvements pending
Lighthouse SEO	Below target	Full metadata, sitemap, robots, structured-data pass pending
Full PWA hardening	Deferred	Offline strategy and authenticated-cache behavior need deliberate design
TypeScript migration	Deferred	Learn and migrate data boundaries incrementally
Tailwind migration	Optional	Re-evaluate if project/team scale makes utility CSS worthwhile

This list should be updated whenever an item moves from:

Deferred
↓
In progress
↓
Implemented
↓
Verified

That makes the documentation a living engineering record rather than a static description.